

MATEMÁTICA COMPUTACIONAL APLICADA

# Aula 07

Álgebra Booleana e suas Aplicações — Parte 2

*De mintermos a mapas de Karnaugh — simplificando fórmulas booleanas visualmente, sem álgebra passo a passo.*

SOP

K-map

don't care

# Nesta aula



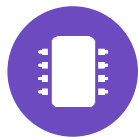
## Forma normal soma de produtos (SOP)

Toda tabela-verdade vira uma fórmula, de forma mecânica.



## Mapas de Karnaugh para simplificar visualmente

Uma grade onde a simplificação aparece a olho nu.



## Aplicações práticas da simplificação

Menos literais, menos portas, menos custo de hardware.

# Onde paramos na Aula 06

Vimos os axiomas booleanos e simplificamos expressões passo a passo, com álgebra pura.

$$xy + x^{-}z + yz = xy + x^{-}z$$

*teorema do consenso — vários passos de álgebra até chegar aqui*



*Hoje: a mesma simplificação, mas "vista" numa grade — sem escrever um único passo de álgebra.*



## O que é um mintermo

Um mintermo é um produto (AND) que vale 1 em exatamente uma linha da tabela-verdade — usando a variável direta quando ela é 1, e complementada quando é 0.

$x = 1, y = 0, z = 1$

mintermo:  $x \cdot \bar{y} \cdot z$

A soma de produtos (SOP) é a soma (OR) dos mintermos de todas as linhas em que a fórmula vale 1.

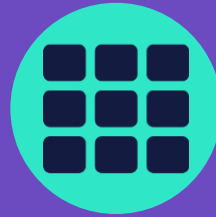
## O XOR, como soma de produtos

| x | y | f |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

f = 1 em (x=0, y=1) e (x=1, y=0), então:

$$f = \bar{x}y + x\bar{y}$$

*Guarde esta fórmula — ela volta no desafio, mais adiante.*



# Mapas de Karnaugh

*Uma reorganização visual da tabela-verdade em que células adjacentes diferem em apenas uma variável — isso permite "ver" simplificações que, na álgebra pura, exigiriam vários passos.*

# Agrupando os 1s

Para  $f(x, y) = x + y$  (equivalente a OR):

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | 0       | 1       |
| $x = 1$          | 1       | 1       |

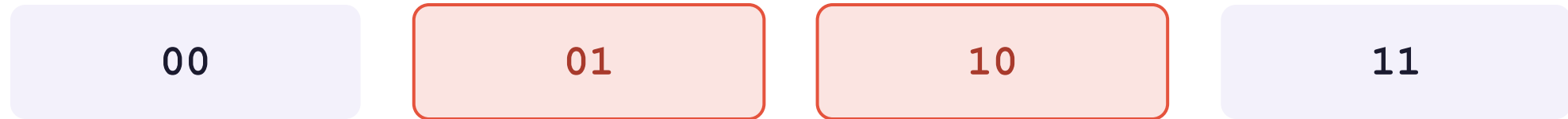
■ linha  $x=1 \rightarrow$  sobra  $x$    ■ coluna  $y=1 \rightarrow$  sobra  $y$

*Os dois grupos, somados, cobrem os três 1s:  $x + y$ .*

## Por que código Gray, e não binário?

Colunas vizinhas no mapa precisam diferir em só uma variável — nem toda ordem garante isso.

ordem binária normal



*01 e 10 parecem vizinhas, mas diferem em 2 bits — a regra do K-map quebra aqui.*

código Gray (usado no K-map)



*cada vizinho troca só 1 bit por vez — inclusive a volta de 10 para 00.*



# Quatro mintermos, um literal

Simplificar  $f(x, y, z) = x\bar{y}z + x\bar{y}z + x\bar{y}z + xyz$ :

| $x \backslash yz$ | 00 | 01 | 11 | 10 |
|-------------------|----|----|----|----|
| $x = 0$           | 0  | 1  | 1  | 0  |
| $x = 1$           | 0  | 1  | 1  | 0  |

O grupo cobre as colunas 01 e 11 (y varia, z é sempre 1) nas duas linhas de x (x também varia).

$$f = z$$

*Bem mais simples que a soma de 4 mintermos original!*

# Adjacência circular

Um K-map é topologicamente um cilindro: a última coluna é vizinha da primeira.

| $x \backslash yz$ | 00 | 01 | 11 | 10 |
|-------------------|----|----|----|----|
| $x = 0$           | 0  | 1  | 1  | 0  |
| $x = 1$           | 0  | 1  | 1  | 0  |



A coluna 10 (última) é adjacente à coluna 00 (primeira) — elas diferem só em  $y$  ( $z$  fica em 0 nas duas). Vale testar esses agrupamentos "que dão a volta".

## Regra prática de agrupamento

1

Agrupe 1s adjacentes em blocos de  $2^n$  células (1, 2, 4, 8...).

2

Prefira os maiores blocos possíveis — menos literais na expressão final.

3

Uma variável desaparece do grupo se ela muda de valor dentro dele; as que não mudam formam o produto daquele grupo.

4

Some (OR) os produtos de todos os grupos necessários para cobrir todos os 1s.



## Condições "don't care"

Às vezes uma combinação de entrada nunca acontece na prática (ex.: um sensor que não pode fisicamente registrar certos valores). Marcamos essas células com X e escolhemos livremente se elas "valem" 0 ou 1 — o que ajuda a formar grupos maiores.

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | 1       | 1       |
| $x = 1$          | 1       | X       |

*Exemplo ilustrativo: se aceitarmos  $X = 1$ , o mapa inteiro vira um único grupo — a expressão colapsa para a constante 1.*

*Um X só é usado a favor da simplificação; nunca é obrigatório cobri-lo.*



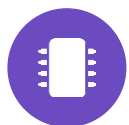
## E com 4 ou mais variáveis?

O K-map cresce para  $4 \times 4$  (16 células) com 4 variáveis, seguindo as mesmas regras — as linhas também passam a se tocar nas bordas, e a topologia vira um torus em vez de um cilindro.

*acima de 5-6 variáveis, o K-map fica impraticável de desenhar à mão*

*É exatamente aí que entram algoritmos como Quine-McCluskey e ferramentas de software — o assunto da próxima seção.*

# Da grade ao mundo real



## Menos literais, menos portas

Cada literal a menos numa expressão simplificada é, em geral, uma porta lógica a menos no circuito final — menor custo, menor consumo, menor chance de falha.



## Escala industrial

Ferramentas de projeto de chips fazem esse tipo de simplificação automaticamente, em escala de milhões de variáveis — o K-map é a versão "à mão" do mesmo princípio.



## Próximo passo do curso

Veremos isso virar fios e portas de verdade na próxima aula de conteúdo.

# SOP e K-map, lado a lado

|                  | Soma de produtos (SOP)                      | Mapa de Karnaugh (K-map)                        |
|------------------|---------------------------------------------|-------------------------------------------------|
| Ponto de partida | Tabela-verdade, linha por linha             | A mesma tabela, reorganizada em grade           |
| Como simplifica  | Álgebra: axiomas e teoremas, passo a passo  | Visual: agrupar 1s adjacentes                   |
| Melhor para      | Poucas variáveis ou provar uma equivalência | 2 a 4 variáveis, simplificar rápido             |
| Limite prático   | Cresce em número de passos                  | Fica difícil de desenhar acima de 5-6 variáveis |

*Na prática, os dois se completam: SOP garante que nada se perde; K-map mostra onde cortar.*



## E se o K-map estiver errado?

*Existe uma forma de conferir uma simplificação feita à mão: bibliotecas de álgebra simbólica, que aplicam os mesmos axiomas automaticamente.*



# Conferindo uma simplificação

Não existe uma biblioteca de álgebra simbólica tão consolidada em C# quanto o SymPy em Python — por isso, só desta vez, o exemplo fica só em Python.

 conferindo\_sympy.py



```
from sympy import symbols
from sympy.logic import SOPform, simplify_logic

x, y, z = symbols("x y z")

# mintermos onde f(x,y,z) = 1, na ordem (x,y,z)
expressao = SOPform([x, y, z], [[0,1,0], [0,1,1], [1,0,1], [1,1,1]])
print(simplify_logic(expressao)) # confere se bate com o K-map feito à mão
```



# Hora de praticar

*Quatro exercícios: montar SOP, dois K-maps e um desafio sobre o XOR.*



## EXERCÍCIO 1

# SOP

Escreva a soma de produtos de  $f(x, y)$  que vale 1 apenas quando  $x = 0$  e  $y = 1$ , ou quando  $x = 1$  e  $y = 0$ . Que operação lógica conhecida é essa?

*SOP:*

---

*Operação conhecida:* \_\_\_\_\_



GABARITO · EXERCÍCIO 1

SOP

| x | y | f |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

SOP:  $x^{-}y + x\bar{y}$  → XOR



## EXERCÍCIO 2

# K-map de 2 variáveis

Monte o K-map de  $f(x, y) = (xy)^{-}$  (NAND) e simplifique visualmente, se possível.

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | ?       | ?       |
| $x = 1$          | ?       | ?       |

*Preencha as células com os valores de  $\text{NAND}(x,y)$  antes de procurar os grupos.*



GABARITO · EXERCÍCIO 2

## K-map de 2 variáveis

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | 1       | 1       |
| $x = 1$          | 1       | 0       |

■ linha  $x=0 \rightarrow \bar{x}$  ■ coluna  $y=0 \rightarrow \bar{y}$

$$\text{NAND}(x,y) = \bar{x} + \bar{y}$$



### EXERCÍCIO 3

## K-map de 3 variáveis

Monte o K-map de  $f(x, y, z)$  que vale 1 nos mintermos  $m_3, m_4, m_5, m_6, m_7$  — ou seja,  $(x,y,z) = (0,1,1), (1,0,0), (1,0,1), (1,1,0)$  e  $(1,1,1)$  — e 0 nas demais. Encontre a expressão simplificada.

| $x \backslash yz$ | 00 | 01 | 11 | 10 |
|-------------------|----|----|----|----|
| $x = 0$           | ?  | ?  | ?  | ?  |
| $x = 1$           | ?  | ?  | ?  | ?  |

*Compare com o exemplo da aula (Slide 9) depois de preencher.*



GABARITO · EXERCÍCIO 3

## K-map de 3 variáveis

| $x \backslash yz$ | 00 | 01 | 11 | 10 |
|-------------------|----|----|----|----|
| $x = 0$           | 0  | 0  | 1  | 0  |
| $x = 1$           | 1  | 1  | 1  | 1  |

■ linha  $x=1 \rightarrow x$  ■ coluna 11  $\rightarrow yz$

$$f = x + yz$$

*5 mintermos viraram 2 termos — mais rico que o exemplo da aula, que virou um literal só.*





#### EXERCÍCIO 4 · DESAFIO

## O XOR contra-ataca

Pegue a expressão original do XOR ( $\bar{x}y + x\bar{y}$ ) e tente simplificá-la com um K-map de 2 variáveis. O que acontece? Por que o XOR é "resistente" à simplificação por agrupamento?

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | 0       | 1       |
| $x = 1$          | 1       | 0       |

*Monte o mapa e procure dois 1s adjacentes — eles existem?*



GABARITO · EXERCÍCIO 4

## 0 XOR contra-ataca

| $x \backslash y$ | $y = 0$ | $y = 1$ |
|------------------|---------|---------|
| $x = 0$          | 0       | 1       |
| $x = 1$          | 1       | 0       |

Os dois 1s ficam em cantos opostos (diagonal) — cada um só tem 0s como vizinhos. Não existe bloco de tamanho 2 (nem maior) para nenhum dos dois.

SOP permanece a forma mais simples:  $x\bar{y} + x\bar{y}$

# Onde estamos no semestre



*Depois de simplificar no papel e na grade, a Aula 08 mostra tudo isso virando fios e portas de verdade.*

# Referências

*Livros — teoria*



**IDOETA, I. V.; CAPUANO, F. G.**

Elementos de Eletrônica Digital. Érica — cap.  
Mapas de Karnaugh.



**TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L.**

Sistemas Digitais: princípios e aplicações.  
Pearson — simplificação por mapa de Karnaugh.



**ROSEN, K. H.**

Matemática Discreta e suas Aplicações. 7. ed.  
AMGH/McGraw-Hill — cap. Álgebra Booleana  
(formas normais e simplificação).

# Documentação e vídeos



## Python — SymPy

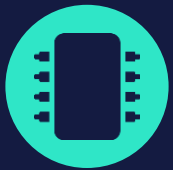
[sympy.logic — simplificação simbólica](#) — SOPform e simplify\_logic.



## Vídeos

[Mapas de Karnaugh — 2, 3 e 4 variáveis](#) — exemplos passo a passo.

[Condições "don't care"](#) — em mapas de Karnaugh.



PRÓXIMA PARADA

Circuitos lógicos

# Do mapa de Karnaugh para fios e portas de verdade.

*Guarde a regra prática de agrupamento — é exatamente o roteiro que vamos seguir para desenhar circuitos com menos portas.*